

操作系统第一次实验报告

李汶峰

202300460158

2023 级网安一班

2025 年 4 月

1 实验目的

2 任务一

2.1 任务描述

任务目标：设计并实现用户态下的两个系统调用，以完成时钟初始化和显示输出功能：

1. 初始化时钟：实现用户态的系统调用以配置系统时钟（SysTick 定时器）。
2. 显示输出：实现用户态的系统调用以输出字符串。

任务要求：

1. 两个系统调用必须同时存在并正常工作。
2. 中断处理过程需尽可能短，减少中断上下文的执行时间。
3. 在用户模式下运行时，原有代码因涉及特权操作（IO 操作和系统寄存器访问）会触发硬故障中断，需通过系统调用在监督模式下执行这些操作。
4. 具体实现需修改 SVC_Handler_Main 函数，处理两个系统调用。

2.2 实验过程

本次实验要求在用户态下实现系统调用，以完成时钟初始化和显示输出。首先在主函数中启动用户模式。此时再次运行原有代码会发现无法成功输出。经过一步步调试发现，输出函数 `print_str()` 调用 `fputc` 函数，而 `fputc` 函数出现 IO 操作，由于在用户模式下不支持 IO 操作，处理器自动产生硬故障中断。同理，`systick_init()` 函数在用户态下也无法访问，需要重新把这两个函数变为系统调用。

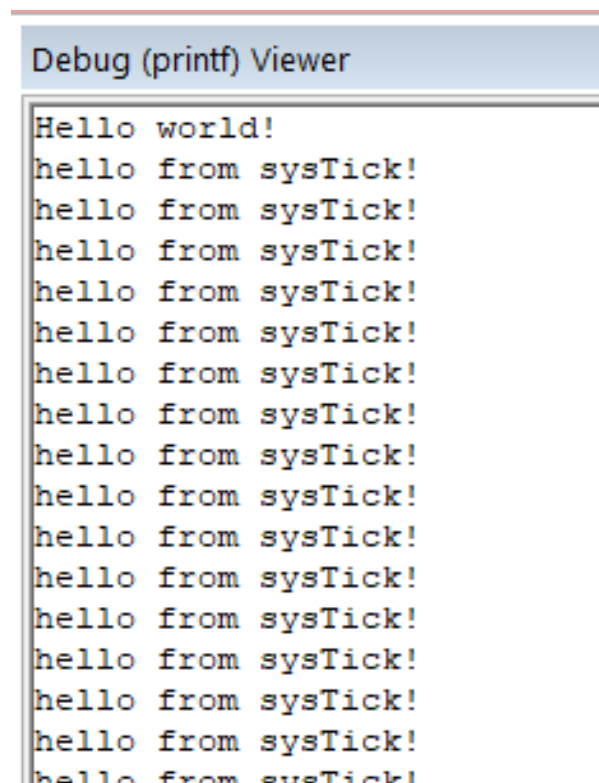
要实现用户态下的时钟初始化和显示输出的系统调用，我们需要修改 `SVC_Handler_Main` 函数，处理两个系统调用：`0x01` 对应显示输出(`syscall_print_str`)，`0x02` 对应时钟初始化(`syscall_systick_init`)。

`syscall_print_str` 函数将输入字符串复制到全局 `buffer` 中，并以标志 `0x01` 触发 SVC 调用，期望处理程序调用 `print_str` 输出字符串。`syscall_systick_init` 函数将计算的时钟频率存入 `buffer`，并以标志 `0x02` 触发 SVC 调用，期望处理程序使用 `SysTick_Config` 配置 SysTick 定时器。具体代码如下：

```
1 void SVC_Handler_Main(int flag)
2 {
3     switch (flag)
4     {
5         case 0x01:
6         {
7             print_str(buffer);
8             break;
9         }
10        case 0x02:
11        {
12            SysTick_Config(*((uint32_t *)buffer));
13            break;
14        }
15    }
16 }
```

2.3 实验结果

实验结果如下图所示，一开始先由系统调用的打印函数打印出 Hello world! 后来系统调用时钟中断函数打印出 hello from sysTick!



```
Debug (printf) Viewer
Hello world!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
hello from sysTick!
```

图 1: 任务一实验结果

3 任务二

3.1 任务描述

修改操作系统与 CPU 有关的源代码，使任务运行在用户态，使用 OSTaskCreate() 建立两个任务，并交替输出。单步调式，重点注意 OSInit()、OSTaskCreate()、OSStart() 和 OSIntExit() 函数，找到可能 unprivileged 模式下不能运行的部分，并修改为系统调用。

3.2 实验过程

通过单步调试发现，当代码运行到 OSStart() 函数后，在进行 OSStartHighRdy() 函数时不能正常进行，会打印出硬中断的报错信息，所以需要将此函数切换为系统调用。

系统调用的思路如下：在 OSStart() 中通过 SWI 0x03 跳转到中断处理程序 SVC_Handler 中。由中断处理程序进行调用函数 SVC_Handler_Main，再跳转到 case 0x03，执行相应的中断处理，修改的代码如下：

```
1 void OSStart (void)
2 {
3     if (OSRunning == OS_FALSE) {
4         OS_SchedNew();
5         OSPrioCur = OSPrioHighRdy;
6         OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
7         OSTCBCur = OSTCBHighRdy;
8         __ASM
9         {
10             SWI 0x03
11         }
12     }
13 }
```

在 SVC_Handler_Main 函数中添加如下代码，实现系统调用：

```
1 case 0x03:
2 {
3     OSStartHighRdy();
4     break;
5 }
```

继续调试，当运行到 OSTimeDly() 函数后，其调用的 OS_Sched() 函数在用户模式下不能正常进行，同样需要将 OS_Sched() 函数切换为系统调用：

```
1 void OSTimeDly (INT16U ticks)
2 {
3     INT8U y;
4     #if OS_CRITICAL_METHOD == 3
5     OS_CPU_SR cpu_sr = 0;
6     #endif
```

```

7   if (OSIntNesting > 0) {
8       return;
9   }
10  if (ticks > 0) {
11      OS_ENTER_CRITICAL();
12      y          = OSTCBCur->OSTCBBY;
13      OSRdyTbl[y] &= ~OSTCBCur->OSTCBBitX;
14      if (OSRdyTbl[y] == 0) {
15          OSRdyGrp &= ~OSTCBCur->OSTCBBitY;
16      }
17      OSTCBCur->OSTCBDly = ticks;
18      OS_EXIT_CRITICAL();
19      __ASM
20      {
21          SWI 0x04
22      }
23  }
24 }

```

在 SVC_Handler_Main 函数中添加如下代码，实现系统调用：

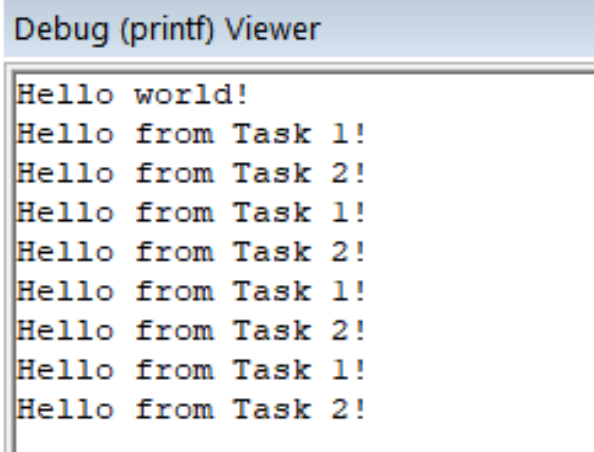
```

1  case 0x04:
2  {
3      OS_Sched();
4      break;
5  }

```

3.3 实验结果

task1 和 task2 进程交替进行并成功交替输出，结果如下：



```

Debug (printf) Viewer
Hello world!
Hello from Task 1!
Hello from Task 2!
Hello from Task 1!
Hello from Task 2!
Hello from Task 1!
Hello from Task 2!
Hello from Task 1!
Hello from Task 2!
Hello from Task 2!

```

图 2: 任务二实验结果

4 任务三

4.1 任务描述

前面的任务中打印函数 `syscall_print_str` 需要事先准备一个 `buffer`，请你试图给出一个不需要 `buffer` 的版本。

4.2 实验过程

当前 `syscall_print_str` 将字符串复制到全局 `buffer`，然后通过 `SVC` 调用（标志 `0x01`）触发 `print_str` 输出。现在进行修改 `user_asm.s`，不需要 `buffer`，用户空间通过寄存器 `R0` 的值保存到 `R6` 中，以保存 `R0` 的内容；在最后将之前保存的 `R6` 移动到 `R1`，作为参数传递给主处理函数。这个操作是将参数传递给 `SVC_Handler_Main` 函数使用。修改部分如下：

```
1 SVC_Handler
2     MOV R6,R0
3     ...
4     MOV R1,R6
5     B     SVC_Handler_Main
```

同时将 `SVC_Handler_Main` 函数的定义修改为：

```
1 void SVC_Handler_Main(int flag,void* str)
```

现在将 `syscall_print_str` 函数修改，直接执行软中断，让其跳转到 `SVC_Handler_Main` 函数之中

```
1 void syscall_print_str(char *str)
2 {
3     __asm
4     {
5         SWI 0x05
6     }
7 }
```

`SVC_Handler_Main` 函数添加新的 `case`，使其通过系统调用打印字符串函数：

```
1 case 0x05:
2 {
3     print_str(str);
4     break;
5 }
```

4.3 实验结果

实验结果如下图所示，修改后实现了不需要 `buffer` 的打印函数的版本。

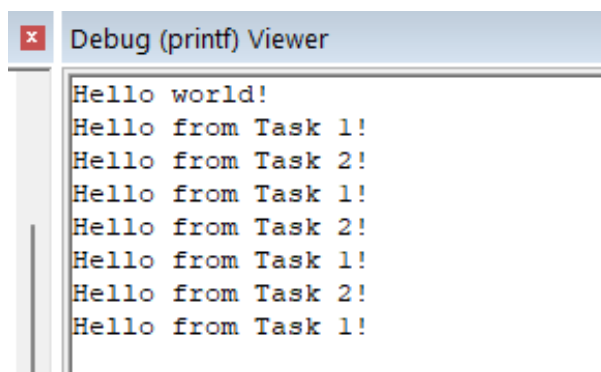


图 3: 任务三实验结果

5 实验感悟

通过本次操作系统实验，我对嵌入式实时操作系统的内核机制有了更深刻的理解，尤其是在 ARM Cortex-M3 架构下实现系统调用、用户态与特权态切换以及多任务调度的过程中，收获颇丰。本次实验以 $\mu\text{C}/\text{OS-II}$ 实时操作系统为平台，围绕用户态与特权态的切换展开，包括系统调用实现、用户态任务调度、优化系统调用三个任务。通过亲手实现系统调用和任务调度，我学会了利用单点调试去找出代码问题所在，跟踪代码状态和寄存器状态，提高了个人代码调试和代码阅读能力，深刻理解了操作系统中的用户态和特权态的问题。